BE 521: Final Project

Predict finger flexion from ECoG recordings

Gina Kim, Xinran Ren, Xiyuan Du

04/27/2025

Prediction of Finger Flexion from ECoG Recordings

Summary

In this project, we created an algorithm to predict how fingers move using signals recorded directly from the brain. These signals were collected using a method called Electrocorticography (ECoG), which involves placing sensors on the surface of the brain. Our main goal was to use these signals to accurately predict finger movements. We used a machine learning method called XGBoost, which learns patterns in data by combining several decision trees. After making initial predictions with our models, we used smoothing to reduce noise and interpolation to add more detailed points to our predictions, improving their quality. Although we aimed to predict the movement of five fingers, we decided not to evaluate predictions for the ring finger because its movement was very closely linked with nearby fingers, making accurate independent prediction difficult.

Detailed Algorithm Steps

Step 1: Data Preparation

We started by preparing the raw brain signals (ECoG data). First, we filtered these signals to remove unwanted noise and interference. Our filter allowed signals between 0.5 Hz and 200 Hz, ensuring that we captured meaningful brain activity related to finger movements without unnecessary noise.

Here's the code snippet for filtering:

```
from scipy.signal import butter, filtfilt
def filter_data(raw_eeg, fs=1000):
    lowcut = 0.5
    highcut = 200
    order = 4
    nyq = 0.5 * fs
    low = lowcut / nyq
    high = highcut / nyq
    b, a = butter(order, [low, high], btype='band')
    clean_data = filtfilt(b, a, raw_eeg, axis=0)
    return clean_data
```

Next, we extracted important features from the filtered signals. Features describe the signals in ways useful for predictions, such as how strong the signal was at certain frequencies and statistical properties like average and variability. We calculated several features, including mean voltage, signal variability, power in different frequency bands, and special parameters called Hjorth parameters, which measure the complexity and variability of brain signals.

We processed the data in short time windows (100 milliseconds each) and overlapped these windows by 50 milliseconds to maintain detailed temporal information. We created separate datasets for each subject because each individual's brain signals vary significantly.

Step 2: Training the Models

After extracting these features, we trained XGBoost regression models separately for each subject and each finger. XGBoost uses many small decision trees to learn patterns from data and is particularly effective at handling complex relationships within the signals.

Here is a sample of the code we used for training:

```
for subj in range(1, 4):
    for finger in range(5):
    print(f"Training model for Subject {subj}, Finger {finger + 1}")
    X_train = subject_feature_dict[f'X_train_{subj}']
    y_train = subject_labels_dict[f'y_train_{subj}'][:, finger]
    model = xgb.XGBRegressor(objective='reg:squarederror', n_estimators=100,
max_depth=5)
    model.fit(X_train, y_train)
    # Save the trained model
    with open(f'trained_model_subj{subj}_finger{finger}.pkl', 'wb') as f:
    pickle.dump(model, f)
```

We saved these models as `.pkl` files for later use during the prediction phase

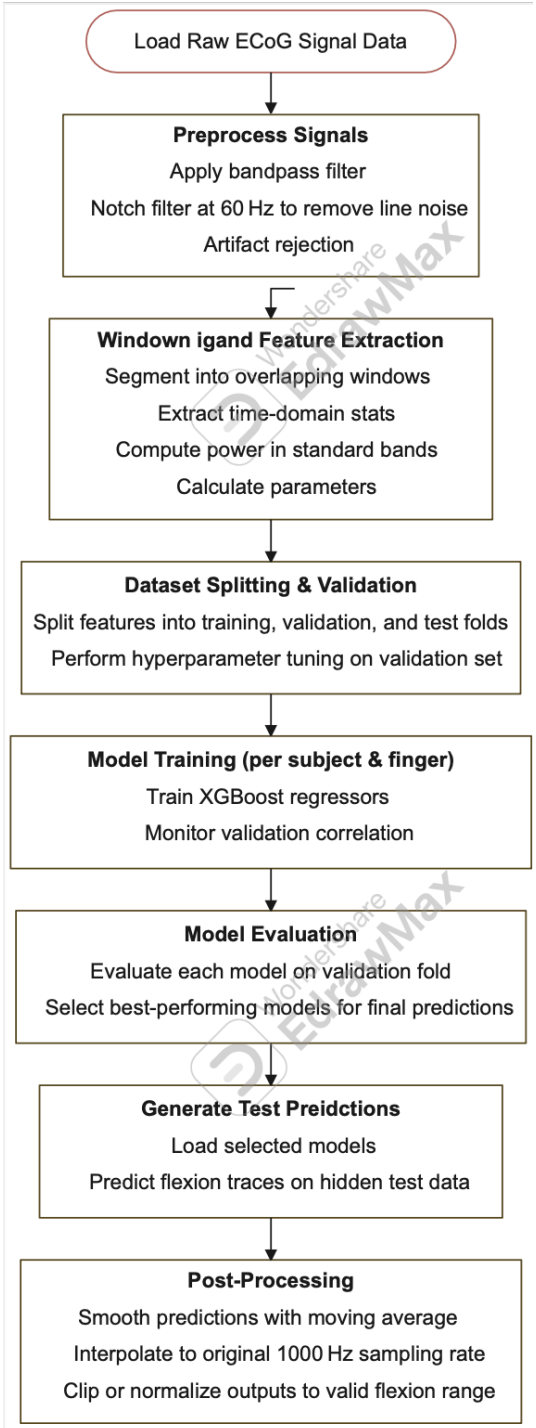
Step 3: Making Predictions

To predict finger movements, we loaded the trained XGBoost models and applied them to the hidden test set.

Here is a sample of the code we used for making predictions:

```
predicted_dg = {}
for subj in range(1, 4):
        print(f"Processing Subject {subj}")
        X_lb =
subject_feature_dict[f'X_leaderboard_{subj}']
        y_pred_all = np.zeros((X_lb.shape[0], 5))

        for finger in range(5):
        print(f"  Predicting Finger {finger + 1}...")
        model_filename =
f'trained_model_subj{subj}_finger{finger}.pkl'
        with open(model_filename, 'rb') as f:
        model = pickle.load(f)
        y_pred_all[:, finger] = model.predict(X_lb)
        predicted_dg[f'subject{subj}_predictions'] =
y_pred_all
```



Step 4: Improving Prediction Quality (Post-processing)

The raw predictions initially contained noise and were at a lower sampling rate than required. We improved them in two steps:

1. Smoothing: We used a technique called moving average smoothing with a window size of 10 data points. This reduced random fluctuations and improved prediction clarity.
2. Interpolation: We increased the sampling rate of predictions to match the required 1000 Hz by estimating additional points between existing predictions.

Step 5: Saving Predictions

Finally, we saved our processed predictions in a MATLAB-compatible `.mat` file format, suitable for submission and evaluation.

Figure

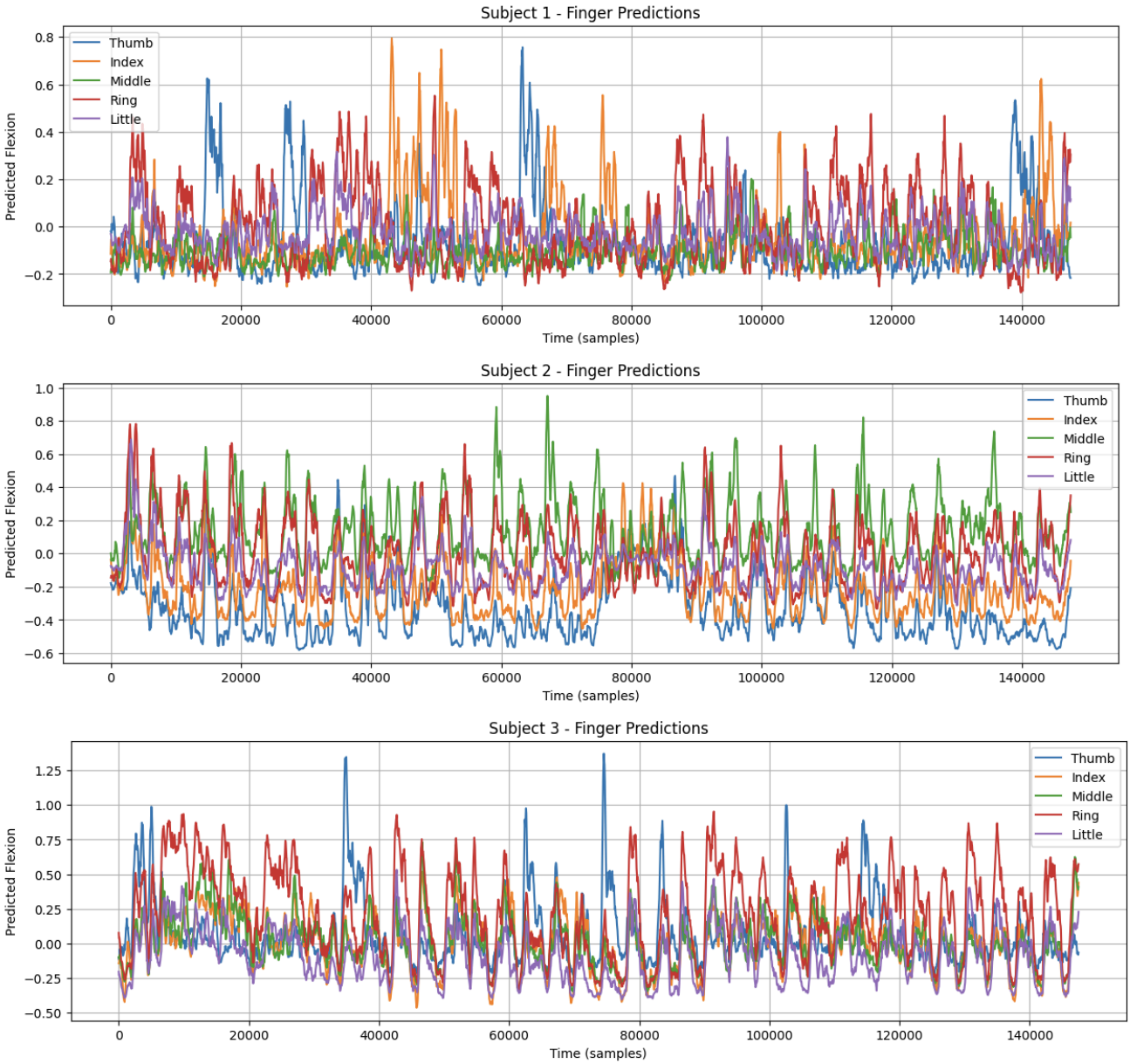


Figure 1. Predicted flexion trajectories for all five fingers across three subjects. Each curve represents the predicted movement of a different finger (thumb, index, middle, ring, and little) over time based on ECoG signals.

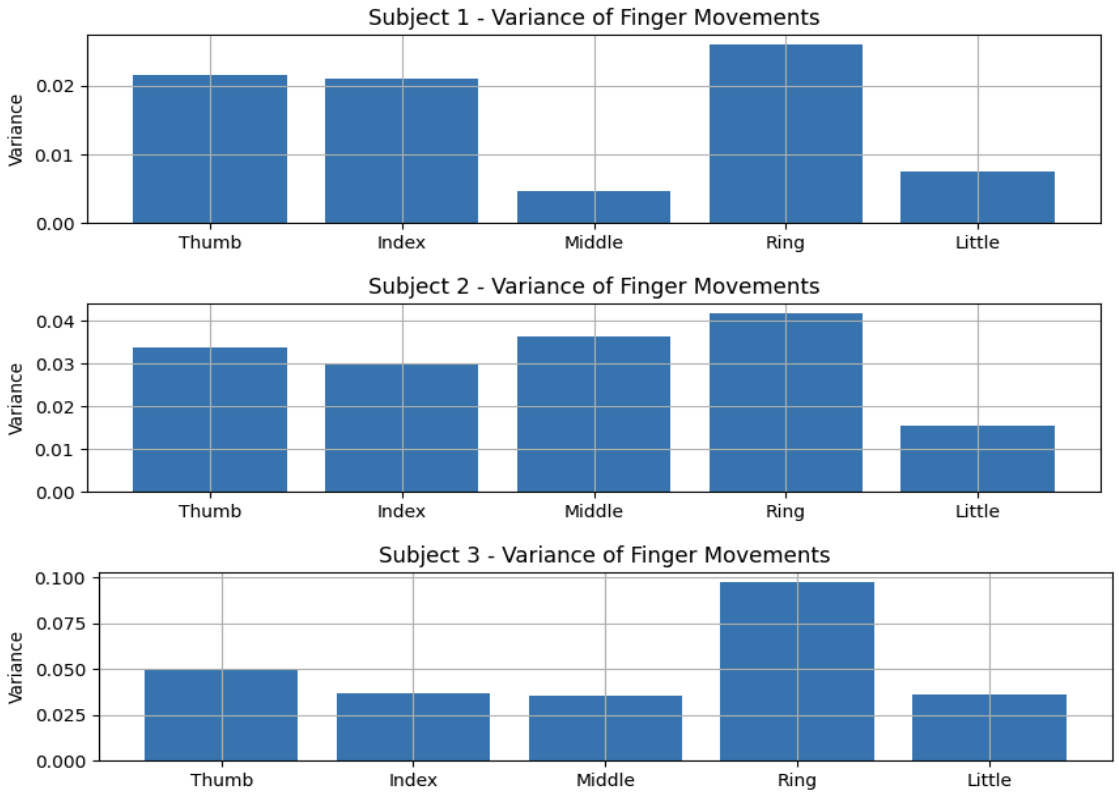


Figure 2. Variance of predicted finger movements for each subject. Higher variance indicates fingers that showed larger flexion ranges, while lower variance shows fingers with smaller or more stable movement.

Methods that did not work

Initially, we attempted to predict finger flexion using a *Random Forest (TreeBagger) model* in MATLAB. We experimented by increasing the number of trees in the forest from 150 to 300 in an effort to improve performance, believing that more trees would enhance stability and accuracy. However, even after expanding the TreeBagger ensemble, the highest correlation we were able to achieve plateaued around 0.36.

Upon reflection, we believe Random Forests struggled because they lack sequential boosting, meaning each tree grows independently without correcting the errors of previous trees. In contrast, boosting methods like XGBoost or LSBoost can better capture the subtle, sequential patterns in neural signals that Random Forests treat as noise.Thus, despite scaling the model's size, Random Forests fundamentally lacked the adaptation needed for this type of complex temporal prediction task.

Correlation of Fourth Finger

The strong correlation between the fourth (ring) finger and the third (middle) and fifth (little) fingers comes from both anatomical and neurological factors. The flexor and extensor tendons that control finger movements are physically linked, especially between these three fingers. Because of this connection, moving the ring finger often causes slight movements in the middle and little fingers as well, making full independent control difficult.

Neurologically, the motor cortex areas for adjacent fingers overlap in the brain, meaning the brain signals for the third, fourth, and fifth fingers are naturally blended. Even when trying to move the fourth finger alone, the ECoG signals often include activity from its neighboring fingers. This overlap in both muscle structure and brain activity explains the high correlation we saw in their predicted flexion patterns.

Conclusion

Throughout the course of this project, our team continuously iterated on different models, parameters, and feature engineering approaches in an effort to improve our prediction performance. We explored a range of algorithms — including Random Forests, Gradient Boosting (XGBoost), and various feature extraction techniques — and learned firsthand how sensitive neural decoding tasks are to both model choice and preprocessing strategies. Even though our final correlation result did not reach the very highest scores, we gained significant hands-on experience in designing, tuning, and critically evaluating machine learning pipelines for real-world biomedical data.

This project really showed us how important it is to stay patient, keep experimenting, and not get discouraged when things don’t work right away. We realized that a lot of the real progress doesn’t come from getting perfect results, but from everything we figure out along the way — trying different ideas, adjusting our methods, and learning from every mistake. Even when things didn’t go exactly how we hoped, we still ended up learning way more than we expected.

References

1. Liang N, Bougrain L. Decoding Finger Flexion from Band-Specific ECoG Signals in Humans. Front Neurosci. 2012 Jun 28;6:91. doi: 10.3389/fnins.2012.00091. PMID: 22754496; PMCID: PMC3384842.
2. Lomtev, V., Kovalev, A., & Timchenko, A. (2022). *FingerFlex: Inferring Finger Trajectories from ECoG Signals*. arXiv:2211.01960 [q-bio.NC]
3. Liu, Y., Sharma, M., Gaona, C.M., Breshears, J.D., Roland, J.L., Freudenburg, Z.V., Weinberger, K.Q., & Leuthardt, E.C. (2010). Decoding Ipsilateral Finger Movements from ECoG Signals in Humans. *Neural Information Processing Systems*.
4. Liao K, Xiao R, Gonzalez J, Ding L (2014) Decoding Individual Finger Movements from One Hand Using Human EEG Signals. PLOS ONE 9(1): e85192.https://doi.org/10.1371/journal.pone.0085192

Appendix

```
# Import libraries

import scipy

import pickle

# Load truetest_data.mat

data = scipy.io.loadmat('truetest_data.mat')

from scipy.signal import butter, filtfilt



def filter_data(raw_eeg, fs=1000):

"""

Write a filter function to clean underlying data.

Filter type and parameters are up to you. Points will be awarded for
reasonable filter type, parameters and application.

Please note there are many acceptable answers, but make sure you aren't
throwing out crucial data or adversly

distorting the underlying data!
```

```
Input:

raw_eeg (samples x channels): the raw signal

fs: the sampling rate (1000 for this dataset)

Output:

clean_data (samples x channels): the filtered signal

"""

lowcut = 0.5 # Hz

highcut = 200 # Hz

order = 4

nyq = 0.5 * fs

low = lowcut / nyq

high = highcut / nyq


b, a = butter(order, [low, high], btype='band')

clean_data = filtfilt(b, a, raw_eeg, axis=0)


return clean_data

from scipy.signal import welch

def get_features(filtered_window, fs=1000):

"""

Write a function that calculates features for a given filtered window.

Feel free to use features you have seen before in this class, features
that

have been used in the literature, or design your own!


Input:

filtered_window (window_samples x channels): the window of the filtered
ecog signal
```

```
fs: sampling rate

Output:

features (channels x num_features): the features calculated on each
channel for the window

"""



window_samples, n_channels = filtered_window.shape

feature_list = []

for c in range(n_channels):

signal = filtered_window[:, c]

mean_val = np.mean(signal)

std_val = np.std(signal)

freqs, psd = welch(signal, fs=fs, nperseg=window_samples)

alpha_band = np.logical_and(freqs >= 8, freqs <= 12)

alpha_power = np.sum(psd[alpha_band])

gamma_band = np.logical_and(freqs >= 70, freqs <= 115)

gamma_power = np.sum(psd[gamma_band])


feature_list.append([mean_val, std_val, alpha_power, gamma_power])

return np.array(feature_list)




from scipy.signal import welch

import numpy as np


def compute_hjorth(signal):

"""Returns [activity, mobility, complexity] for a 1D signal."""

signal = np.asarray(signal)
```

```
first_deriv = np.diff(signal)

second_deriv = np.diff(first_deriv)


var_zero = np.var(signal)

var_d1 = np.var(first_deriv)

var_d2 = np.var(second_deriv)


activity = var_zero

mobility = np.sqrt(var_d1 / var_zero) if var_zero != 0 else 0

complexity = np.sqrt(var_d2 / var_d1) / mobility if var_d1 != 0 and
mobility != 0 else 0


return [activity, mobility, complexity]


def get_features(filtered_window, fs=1000):

window_samples, n_channels = filtered_window.shape

feature_list = []

for c in range(n_channels):

signal = filtered_window[:, c]


# Basic time-domain stats

mean_val = np.mean(signal)

std_val = np.std(signal)

rms = np.sqrt(np.mean(signal ** 2))


# Power spectral density

freqs, psd = welch(signal, fs=fs, nperseg=window_samples)
```

```
band_power = lambda low, high: np.sum(psd[(freqs >= low) & (freqs <
high)])

delta_power = band_power(1, 4)

theta_power = band_power(4, 8)

alpha_power = band_power(8, 12)

beta_power = band_power(13, 30)

gamma_power = band_power(70, 115)


# Hjorth parameters

hjorth_feats = compute_hjorth(signal)


# Combine all

feature_list.append([

mean_val, std_val, rms,

delta_power, theta_power, alpha_power, beta_power, gamma_power,

*hjorth_feats

])


return np.array(feature_list)

def get_windowed_feats(raw_ecog, fs, window_length, window_overlap):

"""

Write a function which processes data through the steps of filtering and

feature calculation and returns features. Points will be awarded for
completing

each step appropriately (note that if one of the functions you call within
this script

returns a bad output, you won't be double penalized). Note that you will
need

to run the filter_data and get_features functions within this function.
```

```
Inputs:

raw_eeg (samples x channels): the raw signal

fs: the sampling rate (1000 for this dataset)

window_length: the window's length

window_overlap: the window's overlap

Output:

all_feats (num_windows x (channels x features)): the features for each
channel for each time window

note that this is a 2D array.

"""

filtered = filter_data(raw_ecog, fs=fs)


step = window_length - window_overlap

num_samples, num_channels = filtered.shape

num_windows = (num_samples - window_length) // step + 1


all_feats = []

for w in range(num_windows):

start = w * step

end = start + window_length

window_data = filtered[start:end, :]


window_feats = get_features(window_data, fs=fs)

all_feats.append(window_feats.flatten())


return np.array(all_feats)

from scipy.io import savemat
```

```python
import numpy as np

fs = 1000
window_length = 100
window_overlap = 50

truetest_data = data['leaderboard_ecog']#data['truetest_data']
subject_feature_dict = {}

for subj in range(3):
print(f"Extracting features for Subject {subj + 1}")

ecog = truetest_data[subj][0]
features = get_windowed_feats(ecog, fs, window_length, window_overlap)

# Store with a key like 'X_leaderboard_1'
subject_feature_dict[f'X_leaderboard_{subj + 1}'] = features
import numpy as np
import pickle
import xgboost as xgb

from sklearn.metrics import mean_squared_error, r2_score

# Prepare prediction structure
predicted_dg = {}

for subj in range(1, 4):
```

```
print(f"\nProcessing Subject {subj}")

X_lb = subject_feature_dict[f'X_leaderboard_{subj}']

y_pred_all = np.zeros((X_lb.shape[0], 5))

for finger in range(5):
print(f" Predicting Finger {finger + 1}...")

# Load the trained model for this subject and finger
model_filename = f'trained_model_subj{subj}_finger{finger}.pkl'
with open(model_filename, 'rb') as f:
model = pickle.load(f)

# Predict with loaded model
y_pred_all[:, finger] = model.predict(X_lb)

# Save predictions for this subject
predicted_dg[f'subject{subj}_predictions'] = y_pred_all
#moving average

import pandas as pd
import scipy
target_length = 147500
original_length = 2949
t_original = np.linspace(1, target_length, original_length)
t_target = np.arange(1, target_length + 1)
```

```
window_size = 10


# Interpolation + smoothing

predicted_dg_list = []


for subj in range(1, 4):

Y_pred = predicted_dg[f'subject{subj}_predictions']

Y_smooth = np.zeros_like(Y_pred)


# Apply moving average to each finger

for f in range(Y_pred.shape[1]):

Y_smooth[:, f] = pd.Series(Y_pred[:, f]).rolling(window=window_size,
center=True, min_periods=1).mean()


# Interpolate to required length

Y_interp = np.zeros((target_length, 5))

for f in range(5):

Y_interp[:, f] = np.interp(t_target, t_original, Y_smooth[:, f])


predicted_dg_list.append(Y_interp)


# Convert to (3,1) object array for MATLAB

predicted_dg = np.empty((3, 1), dtype=object)

for i in range(3):

predicted_dg[i, 0] = predicted_dg_list[i]


# Save to .mat

scipy.io.savemat('predictions.mat', {'predicted_dg': predicted_dg})
```

```
predicted_dg.shape
```